

Ascending into order: A reflective analysis from a small open source development team

Dave Hagan, Owen Watson*, Kirk Barron

Business, Logistics and Information Systems Department, University of Bolton, Deane Road, Bolton BL3 5AB, UK

Abstract

Open Source Software (OSS) has had a profound impact on the way in which software is developed and consequently on the perception of software development. This paper starts with a brief discussion of the eclectic techniques and processes, combinations of which may be used for OSS projects. A ‘real-world’ context will be examined using ‘Reflective Analysis’ focusing on the research and development team present within the Business, Logistics and Information Systems (BLIS) department at The University of Bolton. The reflective analysis explores some assumptions inherent in OSS development. Drawing on findings from experience, industry and background research, an attempt will be made to provide an insight into how in a small team, the use of OSS and associated techniques can lead to the successful development of software. Whilst early results show that smaller OSS projects may appear to have little or no formal structure with closer investigation it is clear this is not the case. Without at least some degree of management, the teams experience indicates that such projects may be prone to failure.

© 2007 Elsevier Ltd. All rights reserved.

Keywords: Open Source Software; Agile development; Reflective analysis; Heterarchy; Development methods

1. Introduction

Open Source Software (OSS) has had a profound impact on the way in which software is developed and consequently on the perception of software development. It has revolutionised the software industry bringing with it new development paradigms and processes for working, extending from the technical to the social to the legal.

In order to better understand the research focus, a brief discussion will be given of the techniques and processes used within OSS development. The methods, if any, used when developing software will be outlined. These areas will be reflected upon with a ‘real-world’ example using the research and development team present within the Business, Logistics and Information Systems (BLIS) department at The University of Bolton. ‘Reflective Analysis’ will be defined to allow an understanding of its contribution to the context of this paper. Drawing on findings from experience, industry and background research, an attempt will be made to

*Corresponding author. Tel.: +44 1204903980.

E-mail addresses: d.hagan@bolton.ac.uk (D. Hagan), orwlis@bolton.ac.uk (O. Watson), kblis@bolton.ac.uk (K. Barron).

provide an insight into how a small team, the use of OSS and associated techniques can lead to the successful development of software.

1.1. *Reflective analysis*

[Miriam-Webster's Online Dictionary](#) defines reflection as “a thought, idea, or opinion formed or a remark made as a result of meditation”. In order to meditate upon our actions we must first understand the ways in which this may be undertaken.

[Schon \(1983, 1987\)](#) defines two methods of reflective practice: Reflection-in-action and Reflection-on-action. Reflection-in-action is a method whereby we may consciously alter our work while we are doing it. For example, if something does not appear to be working correctly, or we are surprised, the problem can be restructured, and the solution used in future actions.

Reflection-on-action is what is generally considered to be reflection. This is looking back on ‘lessons learned’ whereby we evaluate our own process and how we can better prepare for new tasks going forward.

In order for a reflection to be made in the context of the BLIS development team, (‘the team’), at least a minimal analysis of the work undertaken so far may be performed.

Again, [Merriam-Webster's Online Dictionary](#) defines analysis as “an examination of a complex, its elements, and their relations”. From the perspective of the team a reflective analysis, therefore, can be considered as an ongoing evaluation encompassing both of Schon's reflective methods, reviewing processes in place and adapting them for future tasks. This allows for the formation of an ongoing pattern of reflection regarding the progress and experiences of the team.

The reflective analysis will encompass the personal experience of the team and will show how an initial foray into OSS development has progressed over the course of a small development project.

1.2. *Background*

Within the BLIS department at The University of Bolton there is a push towards the development and implementation of OSS. Some of the team are focused upon investigating current techniques for software development focusing on those used within OSS development from both a theoretical and, practical perspective. Through a process of empirical investigation the team are in the process of utilising their findings when developing software.

The University of Bolton has links with the manufacturing industry in its surrounding region. Within ‘Tiger’ economies—such as those within China where manufacturing is inexpensive—the use of OSS allows for a reduction in cost. [Chae and McHaney \(2006\)](#) demonstrates China's commitment to OSS solutions as a way to remove the dependency on Microsoft. Applying a similar solution to manufacturing companies in the UK would potentially allow some of the competitive advantage to be somewhat negated.

From the inception of the Free Software Foundation in 1985 to the emergence of mainstream projects and applications such as Apache HTTPd and the Mozilla suite, OSS development has, over a relatively short period of time, increased greatly in significance ([Warsta & Abrahamsson, 2003](#)). The most prominent project must be the Linux Kernel, with Microsoft's claim that Linux is now their most significant competitor to date ([Kenedy, 2006](#)).

The reason behind the success of such projects is not immediately clear, and is still a topic of ongoing research ([Crowston, 2005](#)). Observing the development process from the standpoint of classical software development techniques it is difficult to see how projects of this nature can succeed ([Mockus, Fielding, & Herbsleb, 2002a, 2002b](#)).

In addition to the techniques used to aid the development of software, the team are investigating the tools and technologies used in application development; in that the team is attempting to develop software using only OSS tools.

2. Challenges

Many authors, among them Polancie, Herieko, and Horvat (2000) claim several risks and limitations concerned with using OSS exist. They cite low code quality, non-existent project plans and non-deterministic stability of the project. However, these types of risks are not universal among OSS development and can exist within proprietary software projects. Although these risks may be inherent with the majority of smaller open source development projects, larger projects tend to be better coordinated (Iannacci, 2005a, 2005b).

Using OSS tools and technologies (in the context of an OSS development culture) carries with it a risk that the software tools will no longer be developed or improved and will simply disappear. With this in mind, the team chose to use software backed by major OSS vendors, gambling with the idea that their projects would continue to be supported and updated. The choice of the Eclipse IDE was a direct result of this, as was the use of Struts Shale, an Apache Software Foundation project.

The main project under development, known as ‘Tabula’, is described in Table 1.

In order to better understand the challenge faced by software developers in this environment, it was felt the best way to learn was to simply ‘jump-in’. That is to say the software development team felt the most productive manner of working was through experimentation and participation. This allowed them to appreciate more readily the challenges faced, and to acquire or enhance new skills, such as that of reflection and collaboration rather than simply read about or observe from a distance (Brown & Duguid, 1991; Lave & Wenger, 1990).

3. Growth and usage of Open Source Software

The growing use of OSS in industrial products (Ruffin & Ebert, 2004) and the maturation of such development projects have meant the opportunity for an improved methodology and, in general, the improved organisation of OSS development. Providing a good example is the successful implementation of OSS solutions in Beaumont Hospital (Fitzgerald & Kenny, 2003).

With OSS becoming more prevalent, large organisations such as IBM, Apple and Sun are becoming increasingly involved and in some cases are becoming project leads and major contributors (Feller & Fitzgerald, 2000). Indeed, academia is taking an increased interest; for example the BCS OSS Special Interest Group.

It can be shown that as an industry OSS is growing. At its current level, the estimated market revenue of OSS is around \$1.8 billion and is expected to rise to \$5.8 billion by 2011 (Kenedy, 2007; Lawton & Notarfonzo, 2007). Large open source companies contribute a large portion of this total, with the likes of RedHat earning \$118.9 million in revenue in the first quarter of 2007 alone (Red Hat Inc., 2007).

The estimated use of OSS within industry is relatively large, with companies surveyed in Germany reporting a 59% adoption rate of OSS, with 80% citing the lower licensing fees (TCO) as a primary reason for adoption (Morris, 2007).

However, with the increase in utilisation of OSS, there has arisen the inevitable inclusion of such software into commercial applications. One of the more controversial areas of commercial OSS adoption is in the area of home entertainment. Companies such as TiVo take components developed as open source and then use them to power their Personal Video Recorders (PVRs). Although TiVo comply with the letter of the OSS license used for the software and contribute their changes back to the community, it is widely felt they do not comply with the ‘spirit’ of the license as it is not possible to run your own version of TiVo’s code on your PVR without TiVo digitally signing it (O’Riordan, 2006). This is something which has angered certain members of the community and with the advent of the GPL v3, provisions have been made to avoid this in the future. (The dangers of TiVoisation and the GPL v3, 2007.)

Continuing the trend of commercialisation of OSS, there is a noticeable ‘grey’ area with resellers who take OSS and package it up, making it more user friendly and easy to use. Linspire sell a Linux distribution branded as their own, offering an alternative to Microsoft Windows (Carmony, 2006); something which has drawn negative feedback from the OSS community.

Yet despite this, the use of OSS in commercial environments indicates a willingness by companies to use OSS within their products.

Table 1
Project summary

<i>Overview</i>	The software under development, named 'Tabula', is a medium sized application for use within the Business, Logistics and Information Systems department at University of Bolton. Its primary role is the collation of staff experience and skill-sets allowing for a more inclusive and efficient project allocation. This will enable the research team to more easily access information when initiating new research projects or areas for investigation. Aside from this, the project was to be used as an investigative tool, exploring new development and management techniques, investigating into OSS development.
<i>Timescale</i>	The project had a relatively loose timescale; although it was required for the software to be available for large scale testing by the end of May 2007. Within this time the developers used their 'design-lead-development' approach (whereby the designer would mock-up screens and then functionality would be programmed around this) so the application design was always one step ahead of the functionality. No deadlines were given for specific components (other than the final end date), but emphasis was given to specific tasks which would receive greater time and attention. This allowed the team a large amount of flexibility while still maintaining a modicum of control over the project. Members could tackle difficult and possibly tedious tasks, and take a break by completing smaller or more interesting problems.
<i>Resources</i>	In terms of resources the development team was severely limited. Despite the requirement for the software to be developed, the availability of machines on which to develop and to test/deploy the software were unavailable for several months, impeding progress significantly. Included among unavailable resources were a lack of adequate access to documentation; the documentation for many technologies being used was at times minimal to non-existent. Again, this was something that was felt severely impacted the progress of the project, yet something which could easily have been avoided.
<i>Technologies</i>	In terms of technology, as briefly discussed, the team opted to use a large proportion of OSS technologies. These included Apache HTTPd, Debian GNU/Linux, Apache Tomcat, Eclipse, MySQL and Postgres coupled with Hibernate, Java, Struts-Shale, etc., with SVN used for version control and Bugzilla used for bug-tracking. However, due to restrictions within the university, development was undertaken with Microsoft Windows as the development platform.
<i>Deliverables</i>	There was a specific requirement for the developed software to be a web-based application. This decision was made for several reasons, including ensuring staff could access the application from outside of the university environment and to allow the application to be tied in with other sections of the university and departmental websites. Due to the use of the application, it is important for managerial and supervisory staff to be able to produce reports and statistics based on the data input.
<i>User benefits</i>	The aim was to provide benefits internally to the department, allowing individual staff to track their project and work history, as well as their current skill-set and publication history. As a department, the benefit is to allow for a more inclusive and efficient project allocation, enabling the research team to more easily access information when initiating new research projects or areas for investigation. As the research within the department is heavily industry focused, there is the aim to allow external organisations to search for specific skill-sets and knowledge before approaching the university. The latter is to be targeted specifically towards SMEs in the manufacturing and creative industry sectors within the North West region.
<i>Issues raised</i>	Over the course of the project, from the inception to the testing phase there were several issues raised. Of these, there are three which are felt to be of particular note. The first issue was the difficulty in acquiring and maintaining the necessary equipment to develop and deploy the software. The second was the lack of adequate or up-to-date documentation for the technologies in use. The third, and perhaps most important issue raised was that of the project management style. The adopted project management style employed, relied upon the members being proactive and self-motivating, however, towards the latter stages of the project it became apparent that one member of the team would have responded better to a traditional management style which ultimately lead to minimal contribution by this member.
<i>Success or failure?</i>	Overall, the project should be considered to be a success for two major reasons. The first reason is due to the successful deployment and testing of the application within the department. There have been few reports of problems and because of the staff awareness of the project throughout its current lifecycle the uptake has been high, with several members offering helpful comments and suggestions for future work. The second is the valuable experience gained by the team in terms of managing a small-scale project of this type and in critically reflecting upon our own and each others work. Along with this, a greater appreciation and understanding of OSS has been gained within the team.

4. Techniques

When looking at OSS development, it can initially appear to have little formal structure. However, for OSS to have progressed as rapidly and have become as successful, a structure of some kind must exist.

Distinct from traditional models, open source projects utilise their own development model; made available by the emergence of global communication technologies that utilise the internet, focused using an open licensing approach (Polancie et al., 2000). As Crowston (2005) has maintained, “[i]n an OSS project, a range of coordination mechanisms seem to be used, but exactly what these practices are has not been studied in detail”.

What has been inferred by Warsta and Abrahamsson (2003) is that the overarching development technique used by OSS appears to be relatively similar to the ‘Agile’ series of methodologies; techniques are focused upon being able to quickly and easily adapt to changing requirements with a minimum of effort.

Although it is the contention that OSS development borrows a lot of its thought processes from the Agile model the OSS paradigm differs from these in several key areas. Cockburn (2002) notes the OSS development paradigm differs from Agile methods in philosophical, economical and team structural aspects.

Both Agile and OSS methods are said to be ‘lightweight’. They remove the need for extensive paper documentation and process handbooks, and do not specify when and where to use a technique (Riehle, 2001).

Key to the OSS philosophy is the rise in the introduction of new methodologies, such as Extreme Programming (XP), introduced by Beck in 1998 (Beck, 1999). Although the ideas behind this are not new, Beck successfully combined them in an innovative manner (Abrahamsson, 2002). Within techniques such as this, as within the OSS community, people are placed at the centre of the process (Riehle, 2001).

OSS moves the development process in a new direction. It uses the global communication strategy and constantly shifting tools, teams and ideas to successfully produce software. It can be said the OSS movement has pioneered a technique for the successful distributed development of software.

5. Coordination

A paper by Iannacci (2005a, 2005b) revealed that within the Linux development project headed by Linus Torvald—the lead developer—there were three coordination processes prevalent. These coordination processes were standardisation, loose coupling and partisan mutual adjustment.

Established processes within these larger projects include a thoughtful list of prerequisites. These include, among others, considerations for organisation, communication, project management, development, documentation and testing. These coordination processes and associated factors are now examined in the context of small-scale development within the Bolton project.

5.1. Standardisation

The ‘Standardisation’ mentioned in the Linux case study by Iannacci (2005a, 2005b) covers a range of aspects within OSS development including a predefined bug reporting format, patch submission routine and mailing list of new features.

The benefits of standardisation within such areas are obvious, they enable a commonality that can be understood by all, ensuring a consistent approach. What is not obvious is the conclusion drawn by Iannacci (2005a, 2005b) which states the inherent loss through context-reliant communication can in some part, be reproduced from these standardised procedures. He affirms that a reproduction of this missing context can be achieved once developers resort to uniform homogenous procedures for problem definition and problem solving (even across spatio-temporal boundaries), provided there is successful social interaction.

Although specified within the Linux development project, there are no predefined standards or formats that are used throughout OSS development, and therefore only generalisations can be made.

Within the OSS development paradigm, it is common for a rough product to be developed quickly and then released. From this point, development continues in the form of bug fixing and change requests from users. A technique referred to as ‘bug-driven development’ (Dietze, 2005) or ‘Bug-Triage’ (Mockus et al., 2002a, 2002b).

With a move towards greater collaboration within BLIS, and as another person dedicated to formally testing software joined, it became apparent that a method of recording any problems encountered was necessary. Rather than implement a formal bug reporting system, the team opted to use a lightweight method for problem recording using Bugzilla, an OSS ‘bug tracking’ application. This strategy, as discussed by Dietze (2002) and Mockus (2002), is prevalent within the OSS community and allows developers and users to report

problems without having to interact directly with project maintainers. Due to the prevalence within the community it was felt appropriate to use this method.

5.2. Loose coupling

The ‘loose coupling’ mentioned in Iannacci’s Linux case study (Iannacci, 2005a, 2005b) relates to another coordination mechanism concerned with workflow management. While suggesting standardisation “pools the developers efforts together and makes them available across spatio-temporal contexts”, Iannacci (2005a, 2005b) describes loose coupling as a coordination of “various social subsystems through weak ties”. The lead developer of the Linux project, Linus Torvalds, himself implies how this loose coupling works, with Iannacci (2005a, 2005b) inferring a heterarchy rather than a hierarchy, where the workflow is

kind of a star [shape], with Linus in the center, surrounded by a ring of lieutenants, and these lieutenants surrounded by a ring of flunkies, maybe the flunkies surrounded by a ring of flunkies’ flunkies ... So that you end up with Linus getting stuff most of the time he doesn’t have to work on very hard, because somebody he trusts has already filtered it. (Moody, 2001)

Iannacci (2005a, 2005b) describes ‘heterarchies’ as nested hierarchies that do not follow the commonly associated top down chains of official command, citing Luhmann’s (1995) description of hierarchy in this context where:

... subsystems can differentiate into further subsystems and that transitive relation of containment within containment emerges.

Mitleton-Kelly (2003) goes on to say that heterarchies can be conceptualised as loosely coupled systems, suggesting parts of a system are a whole in their own right while remaining part of a larger system.

A heterarchy then, indicates a flat system with pivotal actors rather than managers, replacing a vertical alignment of hierarchy with a horizontal plane of cooperation. Pivotal actors within the Linux case study would be Linus Torvalds as the leader, acting as a hinge for his ‘lieutenants’, who themselves are the pivotal actors to others.

OSS has its roots in the origins of software development, where developers would share source code with each other, creating a large community of knowledge where ideas could easily be shared (Warsta & Abrahamsson, 2003). This egalitarian approach compounds the suggestion of a heterarchy where developers reside on an equal plane without any form of management with power over them.

The team within BLIS have adopted a similar egalitarian approach to coordination. The four-member team cannot successfully implement a heterarchical system because of its small size, but do however adopt the principles of pivotal actors working alongside other developers, rather than above them.

As the team initially consisted of only three people, a decision was made to implement no formal leadership and project management structure. Rather, at the inception of each project, the team would decide upon the requirements and overall project aim; following this, a weekly meeting would be held to discuss progress and problems. To facilitate communication within the team a decision was made to set up a Wiki to allow the members to demonstrate their progress.

Within the team, a small initial project was chosen (see Table 1 for details). Following from the earlier discussion, with the team being diminutive in size, it was naively assumed that it would be possible to develop the project with little to no formal documentation and management. It quickly became clear, however, that this process was untenable. After only 3–4 weeks of development work the project had come to a virtual standstill. It was subsequently decided to hold a review to understand the projects failings. During the course of the review it became apparent that the lack of clearly defined goals, standards and project management had lead to each team member harbouring a differing opinion of the direction the project should take, despite the weekly progress meetings. In order to resolve the issues identified, a decision was made to allocate areas of responsibility to different team members. The responsibility was divided between a project management and coordination role and that of a lead developer role in an attempt to stabilise the project.

From the outside, open source projects appear to function in a similar manner, with small teams having loose coordination with no formal structure. However, from the BLIS team’s initial attempt at software

development, it can be seen that without a leader, a project can easily become unmanageable. As with Iannacci's Linux case study (Iannacci, 2005a, 2005b), a project lead need not be a formal one, but can be a more evolutionary one as in the case of informal maintainers who unofficially adopt responsibility of maintaining a patch-set against the main kernel tree (Iannacci, 2005a, 2005b).

In common with Agile techniques, Sharma, Sugumaran, and Rajagopalan (2002) suggest the structure of a project itself is such that the main architects of the project can subdivide tasks to be completed by smaller groups or individuals. In keeping with existing methodologies, the projects have an overall leader and coordinator, along with specific roles allocated to key developers (Dietze, 2003). As is the experience of the BLIS team, without an overall lead, a project is doomed to failure.

5.3. *Partisan mutual adjustment*

Unlike the traditional coordination of teams who work in close proximity, OSS development is, for the most part, a networked set of disjointed developers. Thus, the networked interdependencies are coordinated as a by-product, where work moves through a potentially unbounded set of actors that are based upon, and indeed pivots around, the emergence of social structures. These relationships tend to develop out of a common interest in a particular area of a project, with a select number of developers inadvertently coming into contact with one another. The openness of this development means a critical review can take place by anyone who cares to comment.

The importance of these interactions and the choice to focus on specific areas of interest could be integral to both attracting new developers, and retaining existing ones. As Kim and Boldyreff (2005) states: having many co-developers is essential for an open source project.

Good communication between developers is essential. Taking XP as an example, it is a requirement that the members of a development team are in close proximity to facilitate communication. It is specified for XP, all communication should be verbal. However, the OSS development model dictates developers need not even be in the same country in order to work on a project.

With the proliferation of always-on, fast internet connections, this may be considered something of a non-issue. Developers can effectively communicate in 'real-time' using IRC and can contribute to a Wiki or mailing list detailing their progress.

Unlike the more common distributed communications within OSS development, the team at Bolton were able to adhere to Beck's (1999) suggestion that face-to-face communication was essential. However, as only 60% of the development time accounted for 'face-to-face' communication, the team was operating from both a co-situated and a distributed operational basis.

6. Conclusion

Although there is an apparent need for project management and an overall project lead, due to the failure of design by committee, it is a better strategy to allow management to remain relatively informal. This allows the developers time to fully explore solutions to the problem at hand, and to pursue potentially new and innovative solutions, while remaining within the guidelines defined for the project.

It should be noted the findings of the team so far may not hold true for OSS projects with only a single team member, although when the number of contributors increases, having a management structure in place becomes essential.

OSS development does not appear to have a specific methodology which it can call its own; rather it seems to borrow elements from other development techniques. Perhaps the idea that OSS is a technique of its own is conceivably a misnomer. Rather the techniques themselves are in a constant state of flux, adapting to suit developers needs for any given project.

The idea that OSS follows a specific development cycle could, therefore, be considered to be more by happy accident than by design. Developers choose the method which fulfils their need, and for the majority the choice of a quick release and debug cycle, along with intensive peer review falls close to the border of the Agile movement.

It is suggested by Warsta and Abrahamsson (2003) that OSS is not a development methodology in itself, rather, it is simply a collection of licenses approved by the Open Source Initiative.

When observing smaller OSS projects from an external perspective, it can initially appear they have little or no formal structure. However, although the team is still in the early stages of development, from current experience, and closer investigation it is clear this is not the case. Without at least some degree of management projects are prone to fail, or at least become slow to progress.

As the research team is still in the process of investigating OSS, it has been suggested areas of further study should include a discourse analysis of the interactions between members of the Open Source community, and a deeper investigation into the relationships within OSS teams as well as their structure. Taking this forward, the team will engage further with the Open Source community, recording the interactions and investigating how the community as a whole reacts to the increasing scope of various projects. This will encompass among other details an analysis of the reaction to requests for assistance on expanding projects. This is felt to be an important area for further study, as Boldyreff's investigation (Kim & Boldyreff, 2005) into OSS ERP solutions shows 90% of OSS projects do not expand beyond five team members and the claim is made that projects can sometimes take years before producing any code (Kim & Boldyreff, 2005). However, it is possible to raise a counter to this claim by, for example, looking at the KDE project (KDE is a window manager for Linux). As of version 3.5.1 this has 1053 active developers contributing over 4 million of lines of code, making it larger in terms of lines of code than the Linux kernel itself, which consists of around 3.7 millions lines (Brown & Duguid, 1991).

It is the contention therefore, that OSS development is an evolving process; and in order to gain a greater understanding of the managing of successful projects and to enhance understanding of why projects succeed or fail. Further research is needed on the coordination and management processes required for more effective control of small team OSS development projects. The Bolton team aims to continue its longitudinal study and extend it to other small-scale projects. In addition, the team intend to further document the research through a process of action learning.

References

- Abrahamsson, P. (2002). Agile software development methods: Review and analysis. ISSN 1455-0849 [WWW page] URL <<http://www.i4nf.vtt.fi/pdf/>>; accessed 3.12.2006.
- Beck, K. (1999). Embracing change with extreme programming. *Institute Electrical Electronics Engineers Computer*, 32(10), 70–77.
- Brown, J. S., & Duguid, P. (1991). Organizational learning and communities-of-practice. Toward a unified view of working, learning, and innovation. *Organization Science*, 2(1), 40–57 in Hemetsberger, A., & Reinhardt, C. (2006). Learning and knowledge building in open source communities. *Management Learning*, 37, 187–214.
- Carmony, K. (2006). Interview with Kevin Carmony. [WWW page] URL <http://www.freesoftwaremagazine.com/articles/interview_kevin_carmony_freespire>; accessed 5.12.2006.
- Chae, B. K., & McHaney, R. (2006). Asian trio's adoption of Linux-based open source development. *Source Communications of the ACM Archive*, 49(9), 95–99.
- Cockburn, A. (2002). *Agile software development*. Pearson Education.
- Crowston, K. (2005). A coordination theory analysis of bug fixing in proprietary and free/libre open source software. Working paper. [WWW page] URL <<http://floss.syr.edu/StudyP/050328%20chapter.pdf>>; accessed 12.01.2007.
- Dietze, S. (2003). Improvement opportunities for the open source software development approach and how to utilize them. In *Proceedings of the NetObjectDays 2003*, NODe Transit GmbH.
- Dietze, S. (2005). Agile requirements definition for software improvement and maintenance in open source software development. In *Proceedings of SREP05*, Paris, France.
- Feller, J., & Fitzgerald, B. (2000). A framework analysis of the Open Source Software development paradigm. In *21st international conference on information systems*, Brisbane, Australia.
- Fitzgerald, B., & Kenny, T. (2003). Open source software can improve the health of the bank balance—The Beaumont Hospital experience.
- Iannacci, F. (2005a). Coordination processes in open source software development: The Linux case study. *UK E:CO Issue*, 7(2), 23.
- Iannacci, F. (2005b). Coordination processes in open source software development: The Linux case study. *UK E:CO Issue*, 7(2), 21–31.
- Kenedy, K. (2006). Ballmer: Microsoft to focus on Linux competition, software-as-a-service, internet advertising. [WWW page] URL <<http://www.informationweek.com/news/showArticle.jhtml;jsessionid=CUVTWD1HY5ZKOQSNDBOCKHSCJUMKJVN?articleID=187202415>>; accessed 15.12.2006.
- Kenedy, J. (2007). Standalone OSS market to hit US\$5.8bn. [WWW page] URL <<http://www.siliconrepublic.com/news/news.nv?storyid=ingle8473>>; accessed 18.07.2007.
- Kim, H., & Boldyreff, C. (2005). Open source ERP for SMEs. In *ICMR 2005*, Cranfield University, UK, 6–8 September 2005.

- Lave, J., & Wenger, E. C. (1990). Situated learning: Legitimate peripheral participation. Palo Alto, CA: Institute for Research on Learning. In Hemetsberger, A., & Reinhardt, C. (2006). Learning and knowledge building in open source communities. *Management Learning* 37, 187–214.
- Lawton, M., & Notarfonzo, R. (2007). World-wide open source software business models 2007–2001 forecast: A preliminary review. [WWW page] URL <<http://www.idc.com/getdoc.jsp?containerId=206681>>; accessed 18.07.2007.
- Luhmann, N. (1995). 19pp. “Social systems” in Iannacci, F. (2005). Coordination processes in open source software development: The Linux case study. *UK E:CO Issue*, 7(2), 21–31.
- Merriam-Webster Online. [WWW page] URL <<http://www.m-w.com>>; accessed 11.12.2006.
- Mitleton-Kelly, E. (2003). Ten principles of complexity and enabling infrastructures. In E. Mitleton-Kelly (Ed.), *Complex systems and evolutionary perspectives of organisation: The application of complexity theory to organizations* (pp. 23–50). Oxford, UK: Pergamon.
- Mockus, A., Fielding, R., & Herbsleb, J. (2002a). A case study of open source software development: The Apache server. In *Proceedings of the 22nd international conference on software engineering*, Institute Electrical Electronics Engineers Computer Society.
- Mockus, A., Fielding, R., & Herbsleb, J. (2002b). Two case studies of open source software development: Apache and Mozilla. *ACM Transactions on Software Engineering and Methodology*, 11(3), 309–334.
- Moody, G. (2001). “Rebel Code” in Iannacci, F. (2005). Coordination processes in open source software development: The Linux case study. *UK E: Co I Issue*, 7(2), 21–31 (179pp.).
- Morris, C. (2007). More than half of all German companies use Open Source. [WWW page] URL <<http://www.heise.de/english/newsticker/news/91447>>; accessed 21.07.2007.
- O’Riordan, C. (2006). TiVo-isation explained. [WWW page] URL <http://fsfe.org/en/fellows/ciaran/ciaran_s_free_software_notes/tivoisation_explained_implementation_and_harms>; accessed 18.01.2007.
- Polancie, G., Herieko, M., & Horvat, R. V. (2000). Open source software usage implications in the context of software development. *Informatica*, 29, 483–490.
- Red Hat Inc. (2007). Red Hat reports first quarter results. [WWW page] URL <<http://www.redhat.com/about/news/prarchive/2007/firstquarter.html>>; accessed 21.07.2007.
- Riehle, D. (2001). A comparison of the value systems of adaptive software development and Extreme Programming: How methodologies may learn from each other. In G. Succi, & M. Marchesi (Eds.), *Extreme Programming explained*. Addison-Wesley.
- Ruffin, M., & Ebert, C. (2004). Using open source software in product development: A primer. *Institute Electrical Electronics Engineers software*, 21(1), 82.
- Schon, D. A. (1983). “The reflective practitioner” in Greenwood, J. (1998). The role of reflection in single and double loop learning. *Journal of Advanced Nursing*, 27, 1048–1053.
- Schon, D. A. (1987). “Educating the reflective practitioner” in Greenwood, J. (1998). The role of reflection in single and double loop learning. *Journal of Advanced Nursing*, 27, 1048–1053.
- Sharma, S., Sugumaran, V., & Rajagopalan, B. (2002). A framework for creating hybrid-open source software communities. *Information Systems Journal*, 7–25.
- The Linux Information Project. (2007). The dangers of TiVoisation and the GPL v3. [WWW page] URL <<http://www.lininfo.org/tivoization.html>>.
- Warsta, J., & Abrahamsson, P. (2003). Is open source software development essentially an agile method? In *third workshop on open source software engineering*, Portland, Oregon.

Dave Hagan An experienced software developer within the computer industry for more than 20 years, working with companies including Cisco Systems and Accenture. He is now a senior research fellow at the University of Bolton researching into Open Source Software.

Owen Watson A researcher at the University of Bolton as well as a software developer within university, his research focuses on Open Source Software (OSS) and project management. His background is largely technical holding a B.Sc. and M.Sc. in the field of computing and having worked for a large software company his interest lies in the social and technical reasons for the success of OSS.

Kirk Barron An aspiring HCI practitioner and researcher at the University of Bolton, his research focuses upon Human Computer Interaction within Open Source Software, with a bias towards web application development. Holding a B.Sc. in the computing field and having worked with small software companies, his interest lies in improving software usability.